

Sindh Madressatul Islam University Karachi

Object Oriented Programming



SPRING 2026

Lecture 2 of 16

Engr. Majid Kaleem

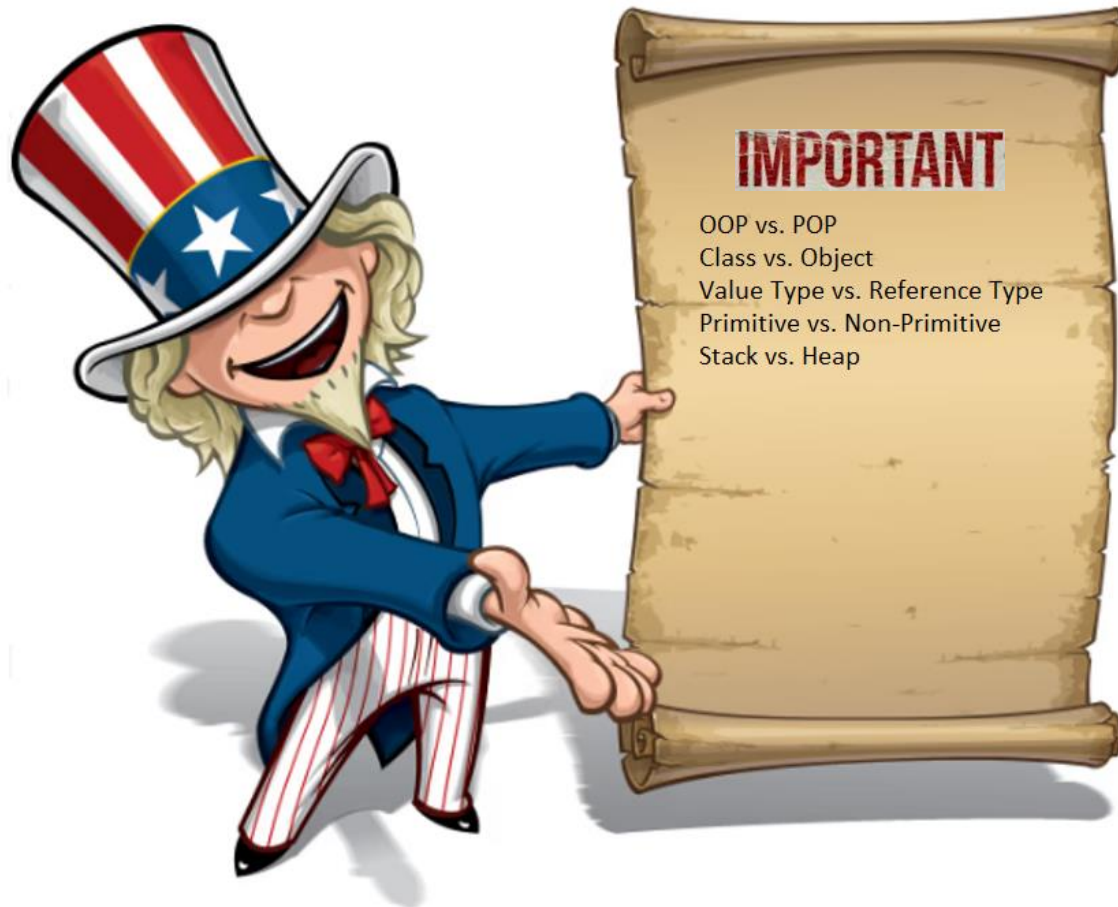
WEEKLY AGENDA

TENTATIVE WEEKLY DATES		TENTATIVE TOPICS
1	16 th Feb – 20 th Feb	INTRO. TO OBJECT-ORIENTED PROGRAMMING, PROCEDURAL VS.OOP, OOP PRINCIPLES, ETC.
2	23 rd Feb – 27 th Feb	CLASSES & OBJECTS
3	02 nd Mar – 06 th Mar	METHODS & MODULAR PROGRAMMING
4	09 th Mar – 13 th Mar	JAVA FUNDAMENTALS ASSIGNMENT #1 & QUIZ #1
5	16 th Mar – 20 th Mar	INHERITANCE
6	23 rd Mar – 27 th Mar	POLYMORPHISM
7	30 th Mar – 03 rd Apr	INTERFACES ASSIGNMENT #2 & QUIZ #2
8	06 th Apr – 10 th Apr	EXCEPTION HANDLING
← MID TERM EXAMINATIONS (13th APR 26 – 17th APR 2026) →		
9	20 th Apr – 24 th Apr	COLLECTIONS FRAMEWORK
10	27 th Apr – 01 st May	FILE HANDLING
11	04 th May – 08 th May	GUI PROGRAMMING
12	11 th May – 15 th May	APPLICATION DEVELOPMENT ASSIGNMENT #3 & QUIZ #3
13	18 th May – 22 nd May	DESIGN PRINCIPLES & PATTERNS
14	25 th May – 29 th May	ADVANCED OOP CONCEPTS
15	01 st Jun – 5 th Jun	CASE STUDIES AND INDUSTRY PRACTICES ASSIGNMENT #4 & QUIZ #4
16	08 th Jun – 12 th Jun	PRESENTATIONS & REVISION WEEK
← FINAL TERM EXAMINATIONS (15th JUN 26 – 19th JUN 2026) →		

Note

- This schedule is subject to revisions as conditions may warrant.*
- Topics will be covered in sequence no matter if the university observes any planned or unplanned holidays.*
- The information in this course outline is subject to revision as conditions may warrant.*

LAST WEEK'S RECAP



NAMING CONVENTIONS IN JAVA

IMPORTANT

Category	Description	Example
Class	Should be (singular) nouns and written in PascalCase (each word starts with a capital letter).	class StudentRecord {...} class AccountManager {...} class CustomerProfile {...}
Interface	Should also follow PascalCase . Interface names often represent capabilities or behaviors.	interface UserAuthenticator {...} interface DataProvider {...} interface PaymentGateway {...}
Method	Method names should typically be verbs or verb–noun combinations written in camelCase .	showStatus() {...} drawCircle() {...} calculateTotal() {...} updateProfile() {...}
Variables	For variables, the Java naming convention is to always start with a lowercase letter and then capitalize the first letter of every subsequent word (camelCase).	String firstName; int timeToFirstLoad; int coffeeSize;
Constant/ enums	UPPER_CASE with underscores	public static final int MAX_USERS = 100; public static final double PI = 3.14159;

```
// Instead of vague names like x, y, z – use meaningful names
public class BankAccount {
    private double balance;

    // Constructor clearly shows purpose
    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    // Method name explains intent: deposit money
    public void deposit(double amount) {
        balance += amount;
    }

    // Method name explains intent: withdraw money
    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Insufficient funds");
        }
    }

    // Method name explains intent: check balance
    public double getBalance() {
        return balance;
    }

    public static void main(String[] args) {
        BankAccount account = new BankAccount(1000);
        account.deposit(500);
        account.withdraw(200);
        System.out.println("Final Balance: " + account.getBalance());
    }
}
```

IMPORTANT

ACCESS MODIFIERS IN JAVA

1. Public:

- A **public** class, method, or variable can be accessed from any other class at any time.
- A class, method, or variable can be declared as public, which means that it is accessible from any class.
- EXAMPLE:
- Like a **public** park — anyone from anywhere can enter and use it freely.

```
public class Animal{  
  
}
```

As such, the Animal class can be accessed by any other class.

```
public int age;  
public int getAge(){  
}
```

ACCESS MODIFIERS IN JAVA

2. Protected:

- Accessible in the same package and subclasses.
- A **variable** and a **method** are the only ones to use the protected access modifier and are closely related to the default access modifier.
- EXAMPLE:
- Like a **family** WhatsApp group — only family members (**subclasses**) and close relatives (**same package**) can access it.

```
protected int age;  
protected String getName(){  
    return "My Name is You";  
}
```

ACCESS MODIFIERS IN JAVA

3. Default:

- Any variable, method, or class declared to use the default access modifier cannot be accessed by any other class outside of the package from which it was declared.
- To use the default access modifier, you simply use **none** of the other access modifiers, and that simply means you are using a default access modifier.
- EXAMPLE:
- Like a **university library** — only students (**same package**) can enter, outsiders cannot.

```
class Bird{  
}
```


ACCESS MODIFIERS IN JAVA

4. Private:

- Allows a variable or method to only be accessed in the class in which it was created.
- No other class beyond the class that created the variable or method can access it.
- EXAMPLE:
- This is closely similar to your internal organs. They are only accessible to the owner.
- Like your **login/password** — only you (*the class itself*) can use it.

```
private void setAge(){  
    System.out.println("Set Age");  
}
```

ACCESS MODIFIERS IN JAVA

- By default, **classes** can only have 2 modifiers:
 1. public
 2. no modifier (default modifier)
- Sometimes, you can embed a class into another class.
- These special classes, inner classes, can be set to private or protected so that only their surrounding class can access it:

```
public class Car {  
    private String brand;  
    private Engine engine;  
    // ...  
    private class Engine {  
        // ...  
    }  
}
```

ACCESS MODIFIERS IN JAVA

IMPORTANT

Access Modifier	Within the Class	Within the Same Package	Subclass/Child Class	In other Packages
Public	Access Allowed	Access Allowed	Access Allowed	Access Allowed
Protected	Access Allowed	Access Allowed	Access Allowed	Access Denied
Default (<i>No Access Modifier</i>)	Access Allowed	Access Allowed	Access Denied	Access Denied
Private	Access Allowed	Access Denied	Access Denied	Access Denied

ACCESS MODIFIERS IN JAVA



- Top-level classes can only be **public** or **default** (no **modifier**).
- Members (fields, methods) can use **all four** modifiers.
- Use **protected** when designing for **inheritance**.
- Use **private** for **encapsulation** — hiding internal details.

CLASSES & PACKAGES

- A package is a namespace that organizes related classes and interfaces together.
- **Purpose:**
 - Avoids name conflicts (two classes with the same name can exist in different packages).
 - Provides access control (via access modifiers).
 - Makes code modular, reusable, and easier to maintain.
 - Think of a package like a folder in your computer that groups related files.
 - i.e., **packages** = **folders** that organize related classes for modularity, readability, and access control.

CLASSES & PACKAGES

- **Package:**
 - A package can contain any number of public classes.
 - **Example:** package university; can have ***Student.java***, ***Teacher.java***, ***Course.java*** — all declared public.
 - Each class lives in its **own file**.
- **File:**
 - In a single .java file, you can only have **one public class**, and the file name must match that class.
 - **Example:**
 - Student.java → contains public class Student { }
 - Teacher.java → contains public class Teacher { }
 - Other non-public classes can coexist in the same file, but only one can be public.

- Student.java

```
package university;

public class Student {
    private String name;

    public Student(String name) {
        this.name = name;
    }

    public void showName() {
        System.out.println("Student: " + name);
    }
}
```

- Teacher.java

```
package university;

public class Teacher {
    private String subject;

    public Teacher(String subject) {
        this.subject = subject;
    }

    public void showSubject() {
        System.out.println("Teaches: " + subject);
    }
}
```

- Main.java

```
import university.Student;
import university.Teacher;

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Ali");
        Teacher t = new Teacher("Mathematics");

        s.showName();
        t.showSubject();
    }
}
```


- Multiple Classes in One File:

```
// File: School.java
package university;

public class School {
    public static void main(String[] args) {
        Student s = new Student("Alice");
        Teacher t = new Teacher("Math");

        s.showName();
        t.showSubject();
    }
}

class Student {
    private String name;

    Student(String name) {
        this.name = name;
    }

    void showName() {
        System.out.println("Student name: " + name);
    }
}

class Teacher {
    private String subject;

    Teacher(String subject) {
        this.subject = subject;
    }

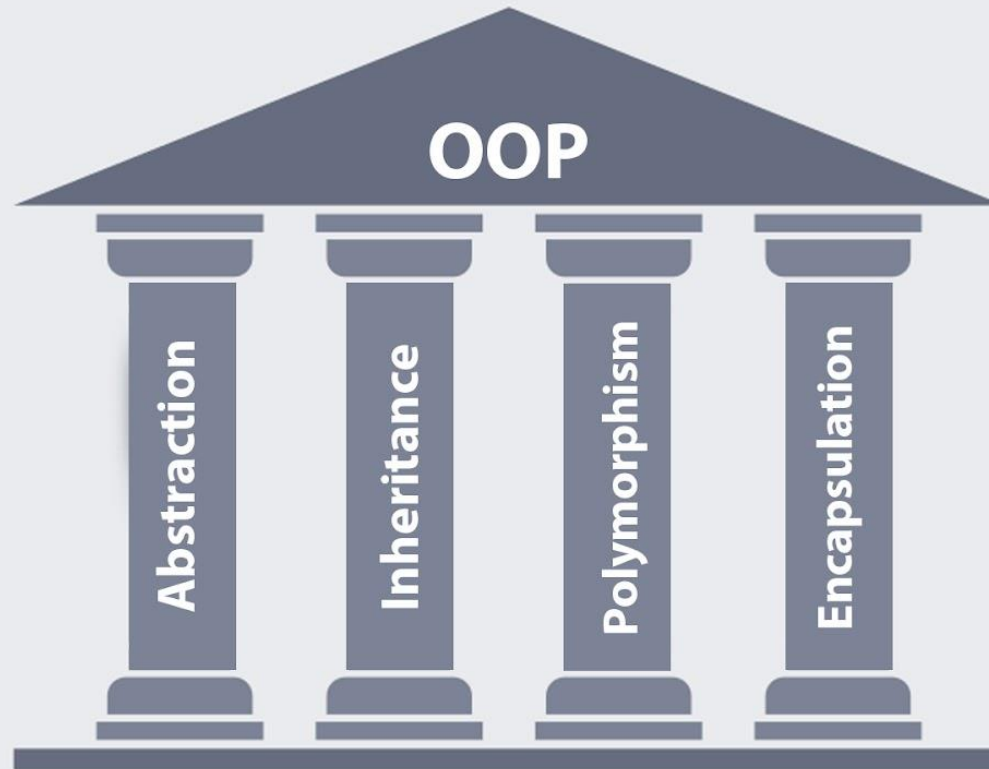
    void showSubject() {
        System.out.println("Teacher teaches: " + subject);
    }
}
```

CLASSES & PACKAGES

- You can write multiple classes in a single **.java** file, but there are some important rules:
 1. **Only one public class per file**
 - If a class is declared **public**, the **file name** must match that **class name**.
 2. **Other classes in the same file**
 - You can declare additional classes in the same file, but they **must not be public**.
 - They will have package-private (default) access, meaning they're only accessible within the same package.
 3. **Best practice**
 - Although allowed, it's **not recommended** to put **multiple classes** in **one file**.
 - For clarity and maintainability, Java convention is **one** top-level class per file.

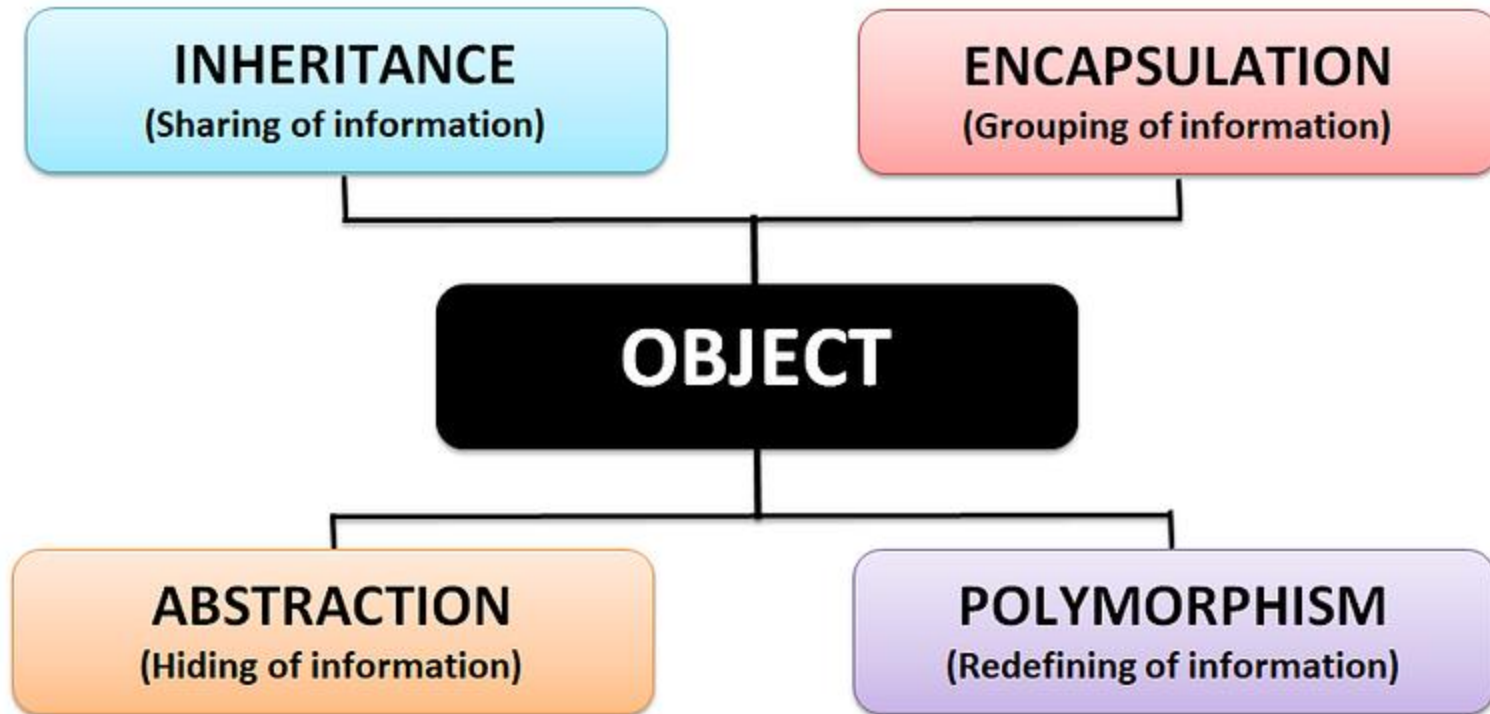
FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

The Four Pillars



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

EXAMPLE



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

1. Encapsulation – *The Technique of Data Hiding and Bundling*

- **Definition:** Wrapping data (attributes) and methods (functions) together inside a class, while controlling access to them.
- **Real-life example:**
 - *Think of a capsule medicine:* the ingredients are hidden inside, and you only interact with the capsule as a whole.
 - *In software:* a Bank Account class hides the balance and only allows deposits/withdrawals through defined methods.

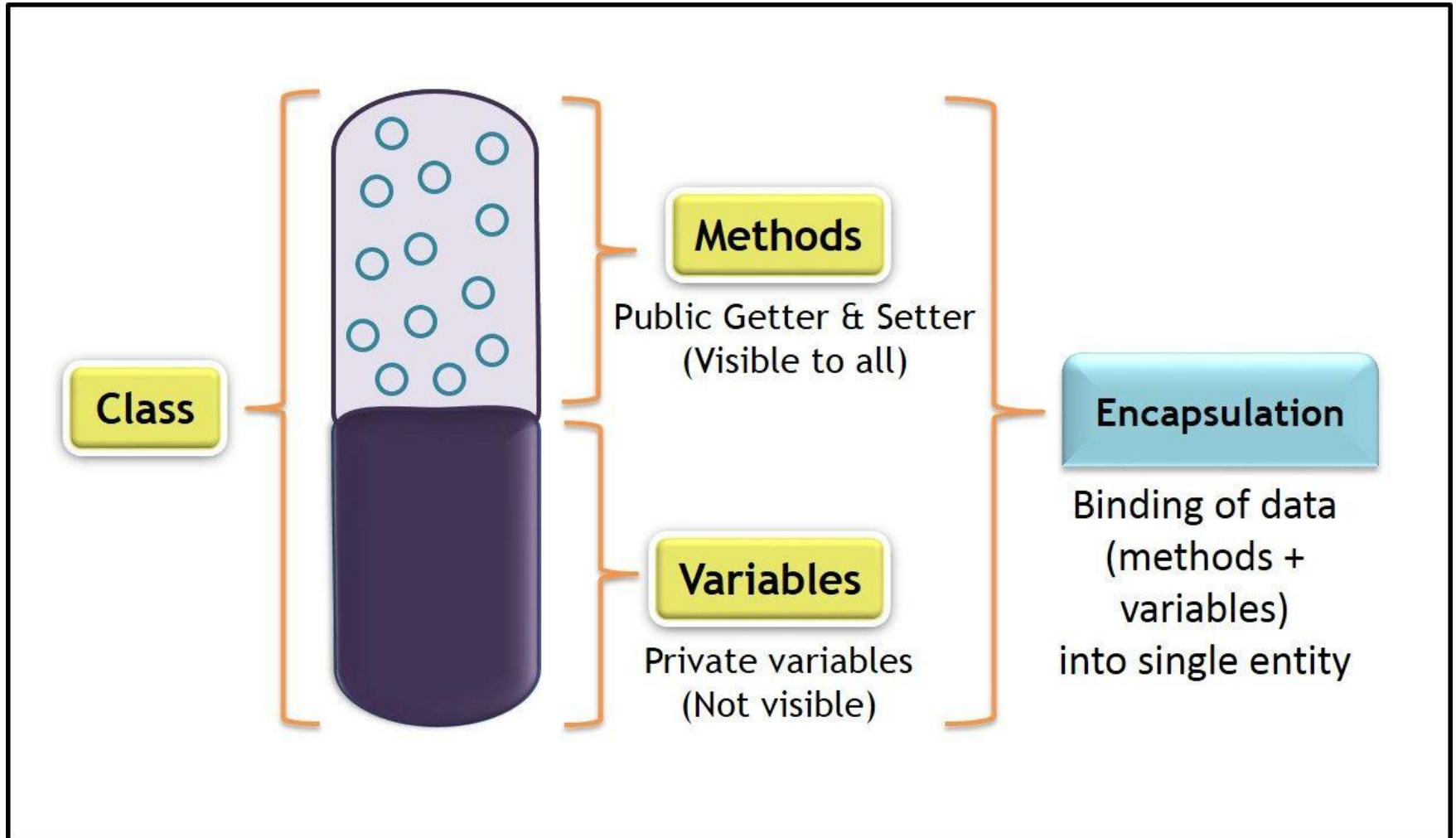
FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

1. Encapsulation – *The Technique of Data Hiding and Bundling*

- It is the process of combining data (attributes) and the methods (functions) that interact with it into a single class.
- Data hiding is the main objective of encapsulation.
- To block direct access from outside the class, data members can be set to private.
- As a result, public methods known as 'getters' (for obtaining data) and 'setters' (for changing data) are used to allow access.
- This restricted access guarantees data integrity and prevents unwanted modifications.

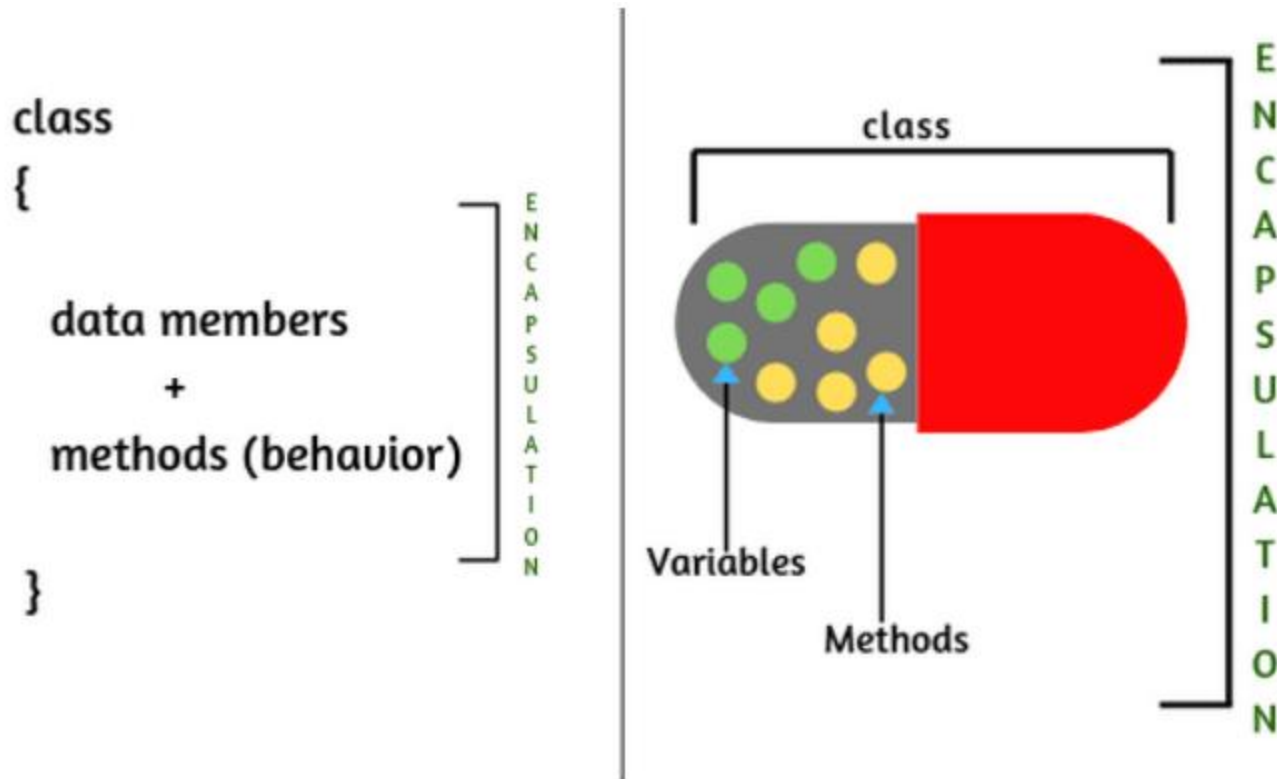
FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

1. Encapsulation – *The Technique of Data Hiding and Bundling*



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

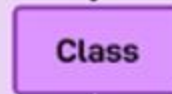
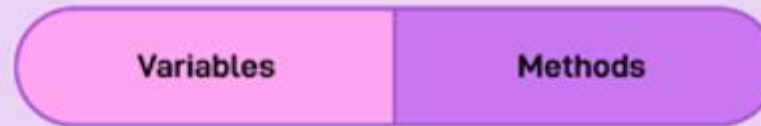
1. Encapsulation – *The Technique of Data Hiding and Bundling*



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

Encapsulation

- Process of wrapping code and data together into a single unit. It means each object in the code should control its own state



```
public class Car {  
    // private variable  
    private String name;  
  
    // getter method for name  
    public String getName() {  
        return name;  
    }  
  
    // setter method for name  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Diagram illustrating the encapsulation of variables and methods within a class. The code defines a `Car` class with a private variable `name` and two methods: `getName()` (getter) and `setName()` (setter). Arrows point from the `private String name;` line to a "Variables" label and from the two method definitions to a "Methods" label.

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

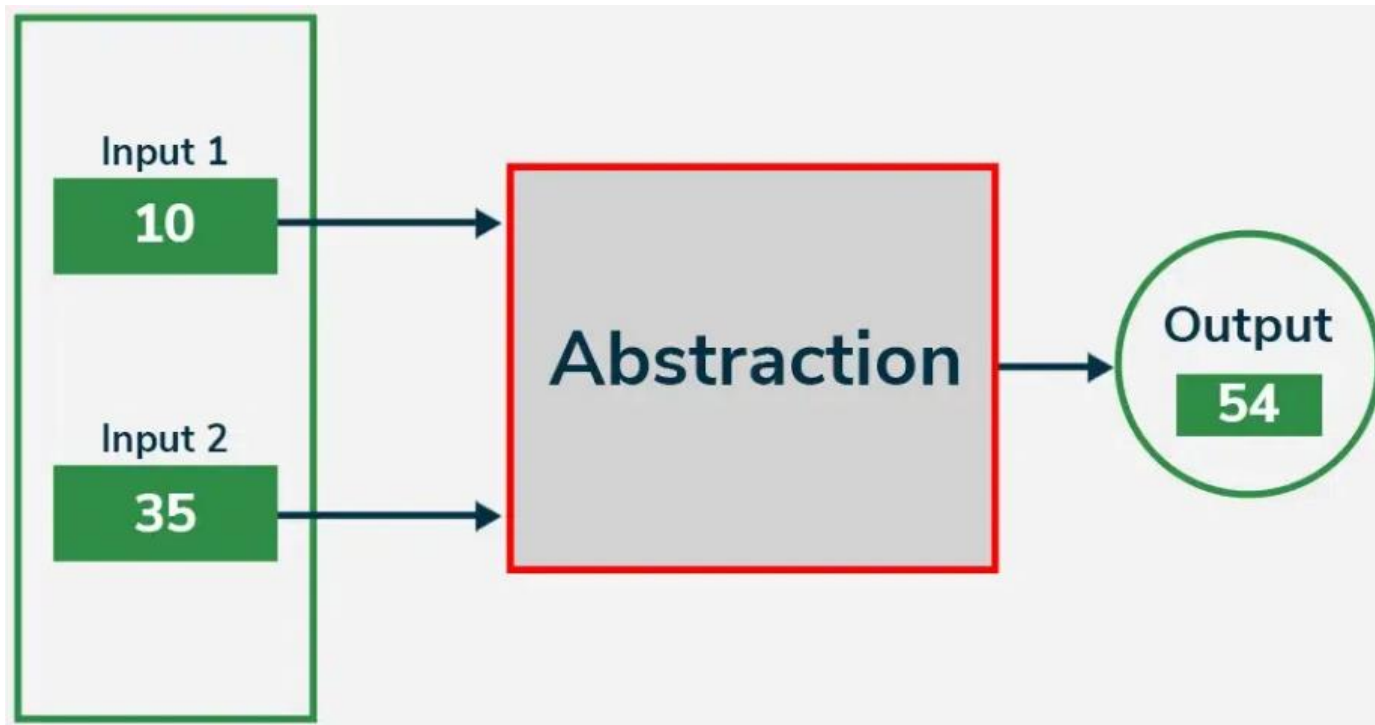
2. Abstraction: – *Reducing Complexity by Hiding Details*

- *Definition:* Hiding complex implementation details and showing only the essential features.
- *Real-life example:*
 - *Driving a car:* you use the steering wheel, accelerator, and brakes without knowing how the engine or transmission works internally.
 - In software: a Payment Gateway API lets you call `makePayment()` without exposing the internal encryption or transaction handling.

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

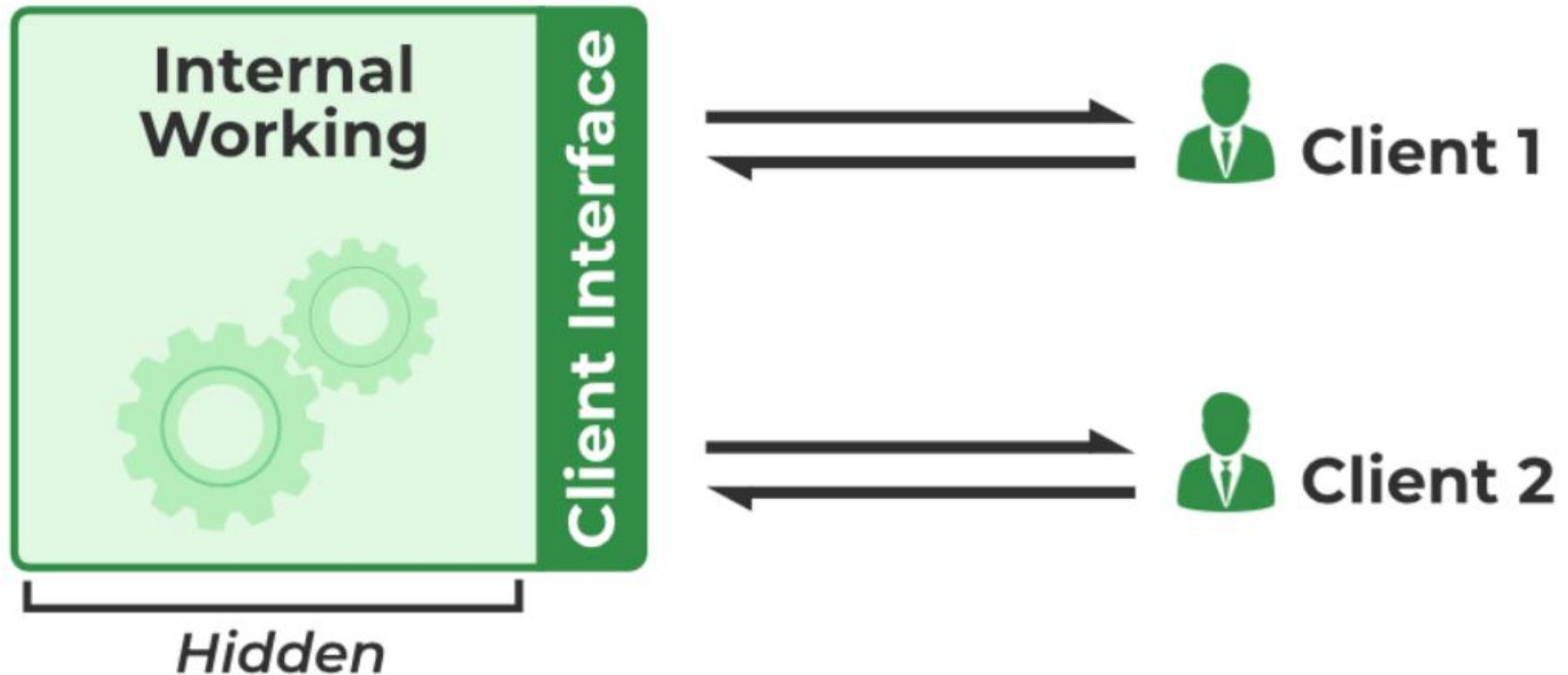
2. Abstraction: – *Reducing Complexity by Hiding Details*

- Hiding complex implementation details and showing only essential features, focusing on **what** an object does rather than **how** it does it (e.g., using a car's accelerator without knowing engine mechanics).



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

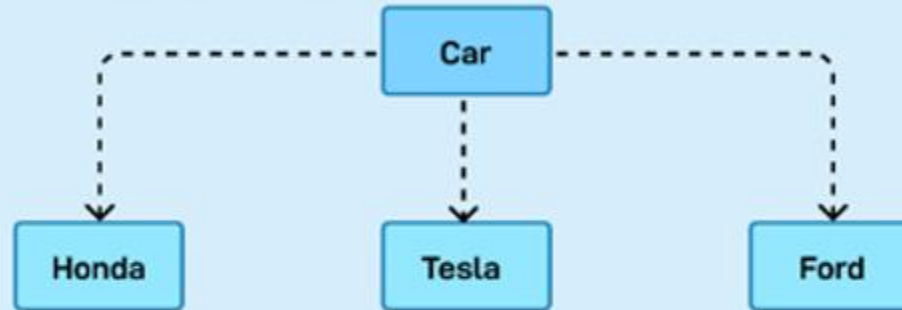
2. Abstraction: – *Reducing Complexity by Hiding Details*



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

Abstraction

- Process of hiding implementation details and exposing only the functionality to the user.



```
public abstract class Car {  
    public abstract void stop();  
}  
  
// Concrete class  
public class Honda extends Car {  
    @Override public void stop() {  
        System.out.println("Honda::Stop");  
        System.out.println(  
            "Mechanism to stop the car using break");  
    }  
}
```

Abstract Class and Method

Providing Implementation

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

3. Inheritance: – *Expanding Current Foundations*

- **Definition:** A mechanism where one class *derives* properties and behaviors from another.
- **Real-life example:**
 - A child inherits traits (like eye color or surname) from parents.
 - In software: a Dog class can inherit from an Animal class, gaining attributes like eat() and sleep() while adding its own bark().

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

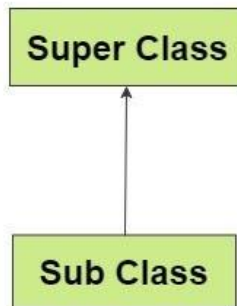
3. Inheritance: – *Expanding Current Foundations*

- OOP's ability to promote *reusing code* through inheritance is one of its most powerful characteristics.
- A new class (subclass or child class) may obtain properties and behaviors from an existing class (superclass or parent class), like children inherit characteristics from their parents.
- This establishes a '**is a**' relationship as in 'Dog **is an** Animal' or 'Car **is a** Vehicle.'

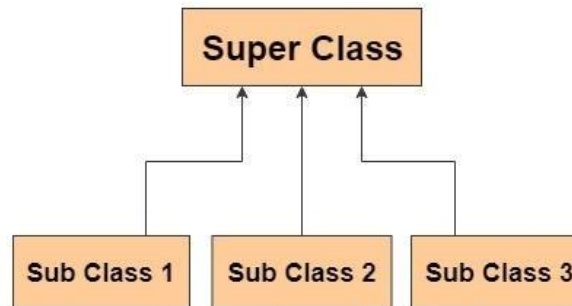
FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

3. Inheritance: – *Expanding Current Foundations*

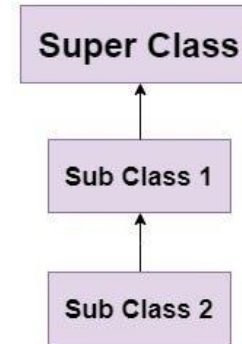
Single Inheritance



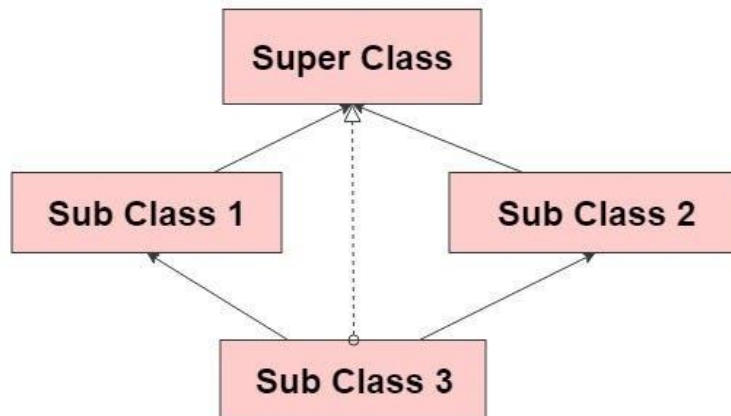
Hierarchial Inheritance



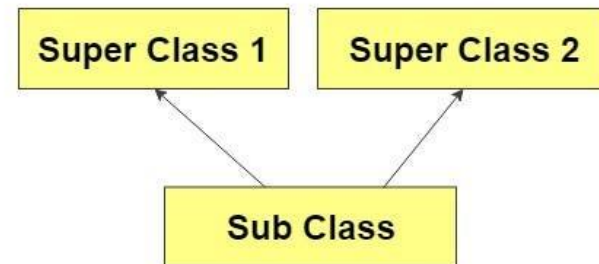
MultiLevel Inheritance



Hybrid Inheritance

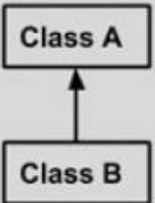
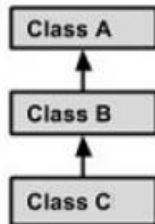
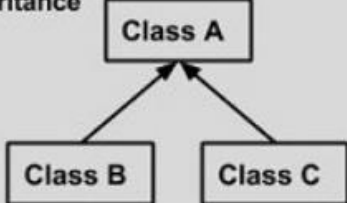
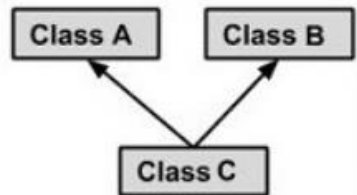


Multiple Inheritance



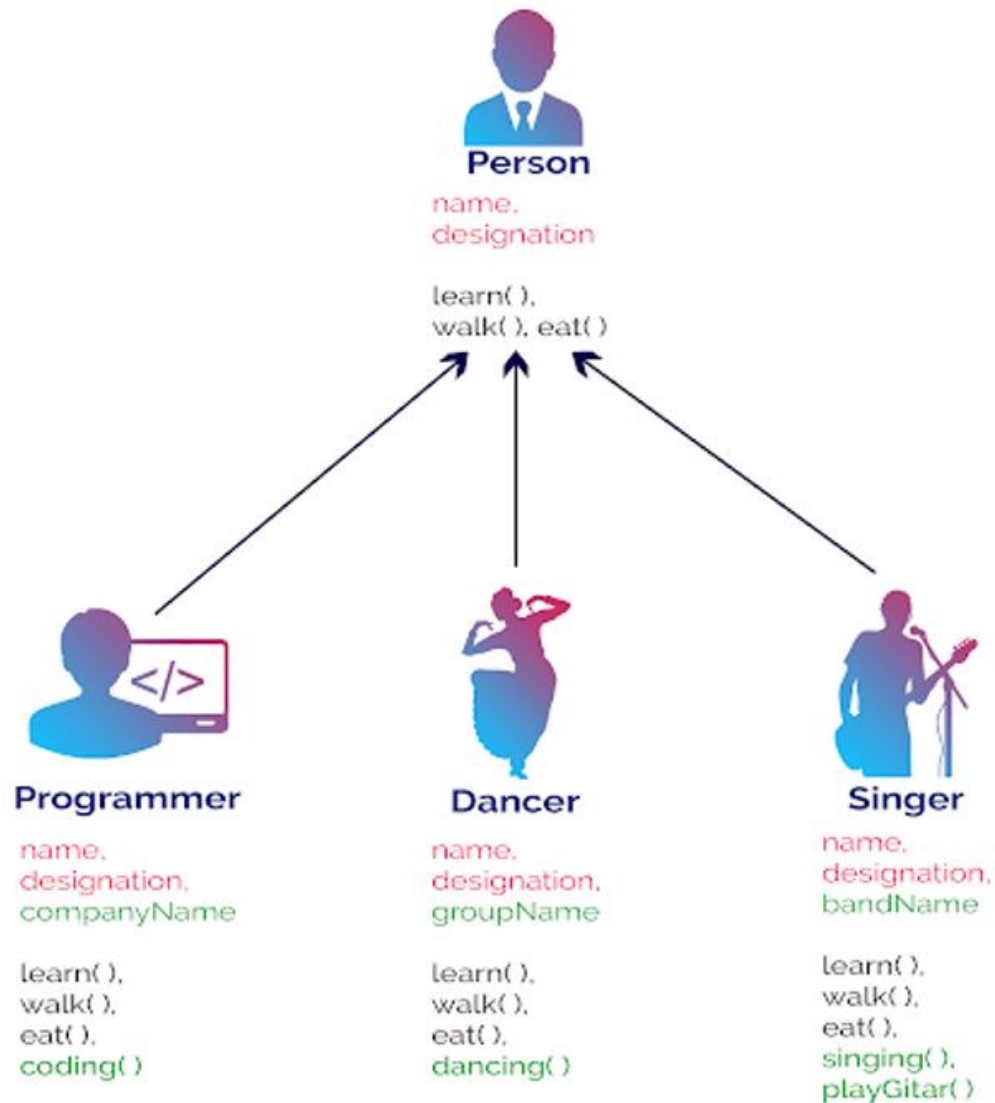
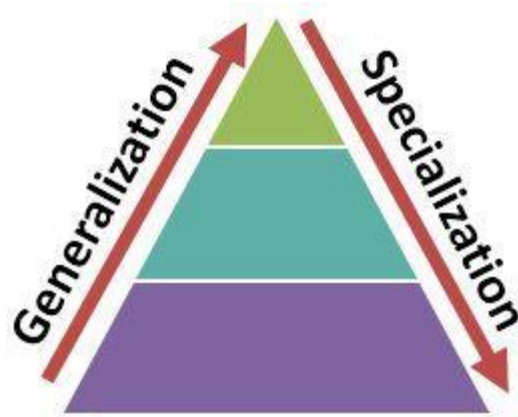
FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

3. Inheritance: – *Expanding Current Foundations*

Single Inheritance  <pre>graph BT; B[Class B] --> A[Class A]</pre>	<pre>public class A { } public class B extends A { }</pre>
Multi Level Inheritance  <pre>graph BT; C[Class C] --> B[Class B]; B --> A[Class A]</pre>	<pre>public class A {} public class B extends A {.....} public class C extends B {.....}</pre>
Hierarchical Inheritance  <pre>graph BT; B[Class B] --> A[Class A]; C[Class C] --> A</pre>	<pre>public class A {} public class B extends A {.....} public class C extends A {.....}</pre>
Multiple Inheritance  <pre>graph BT; A[Class A] --> C[Class C]; B[Class B] --> C</pre>	<pre>public class A {} public class B {.....} public class C extends A,B { } // Java does not support mutiple Inheritance</pre>

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

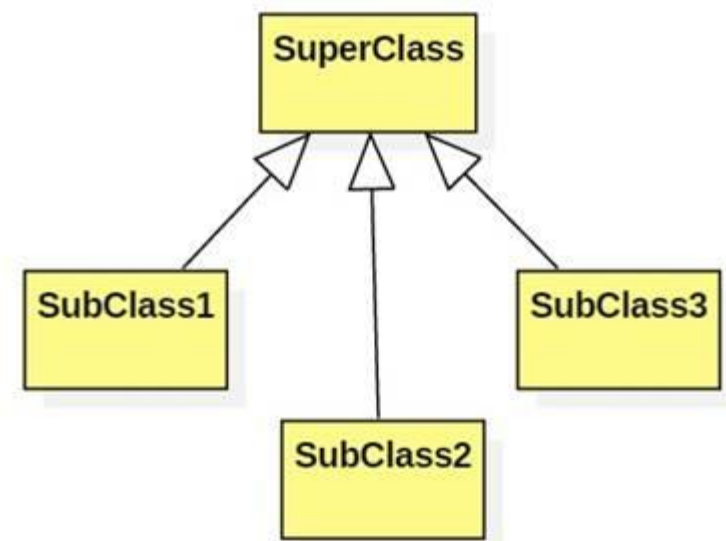
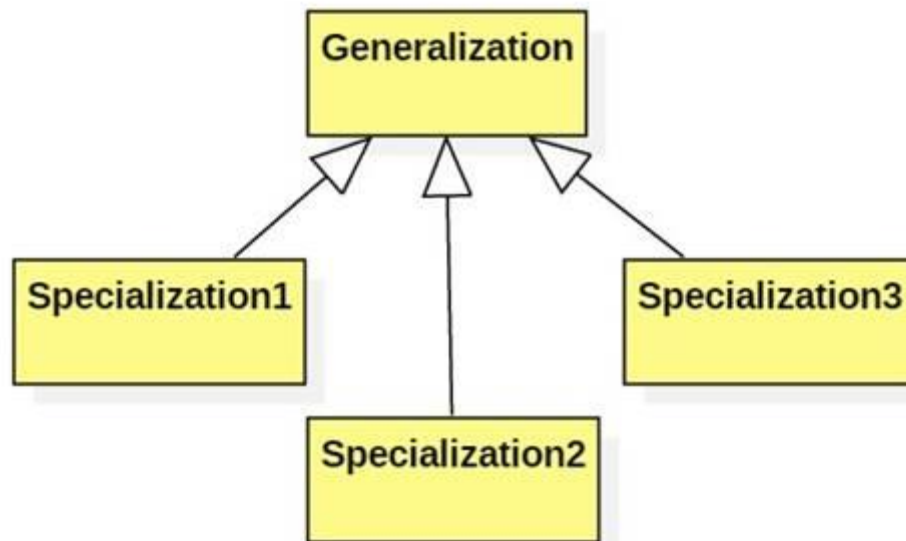
3. Inheritance: – *Expanding Current Foundations*



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

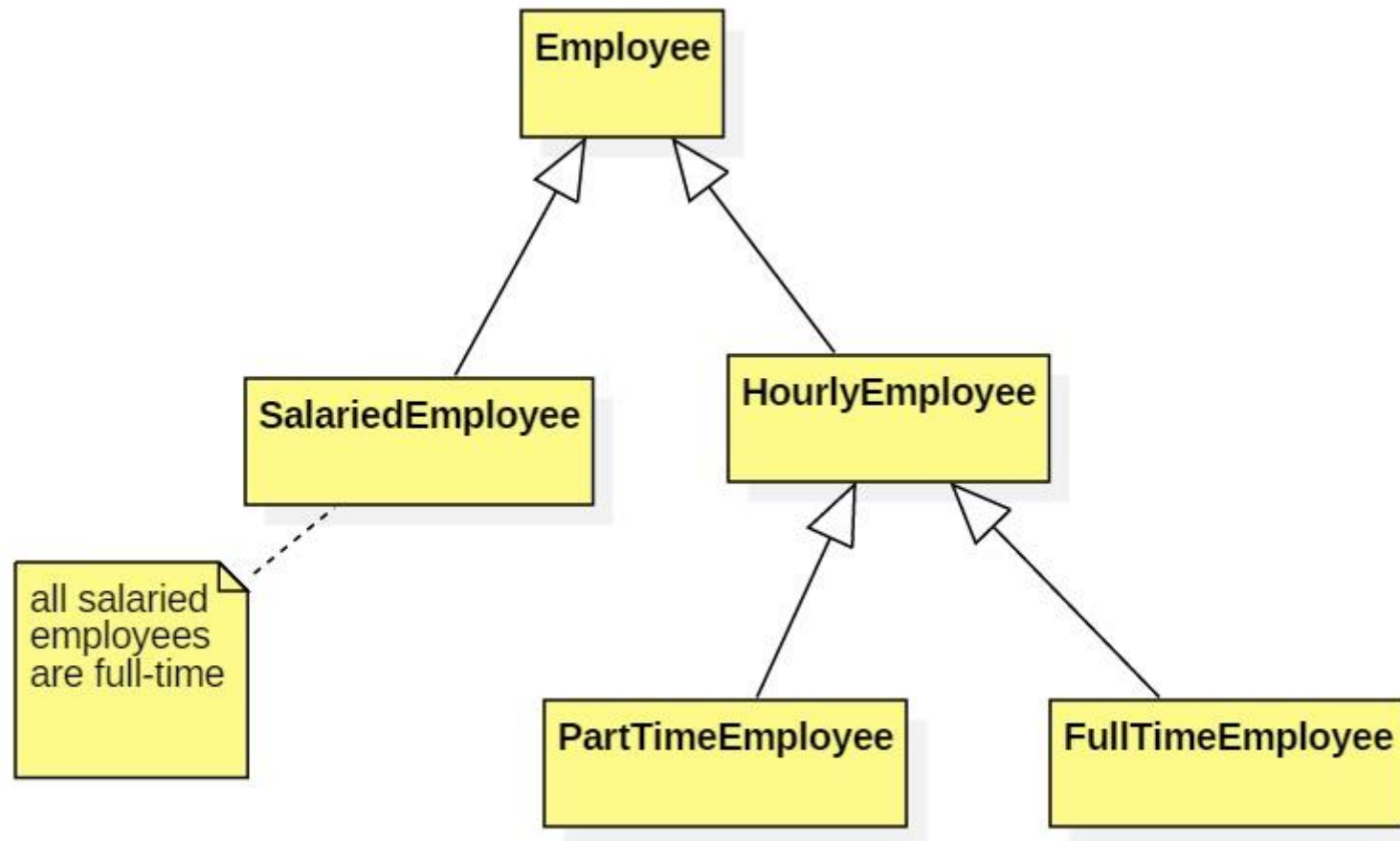
3. Inheritance: – *Expanding Current Foundations*

- Parent (**Super Class**): Represents generalization. It captures what is common across all subclasses.
- Subclass (**Child Class**): Represents specialization. It refines or extends the parent class with unique features.

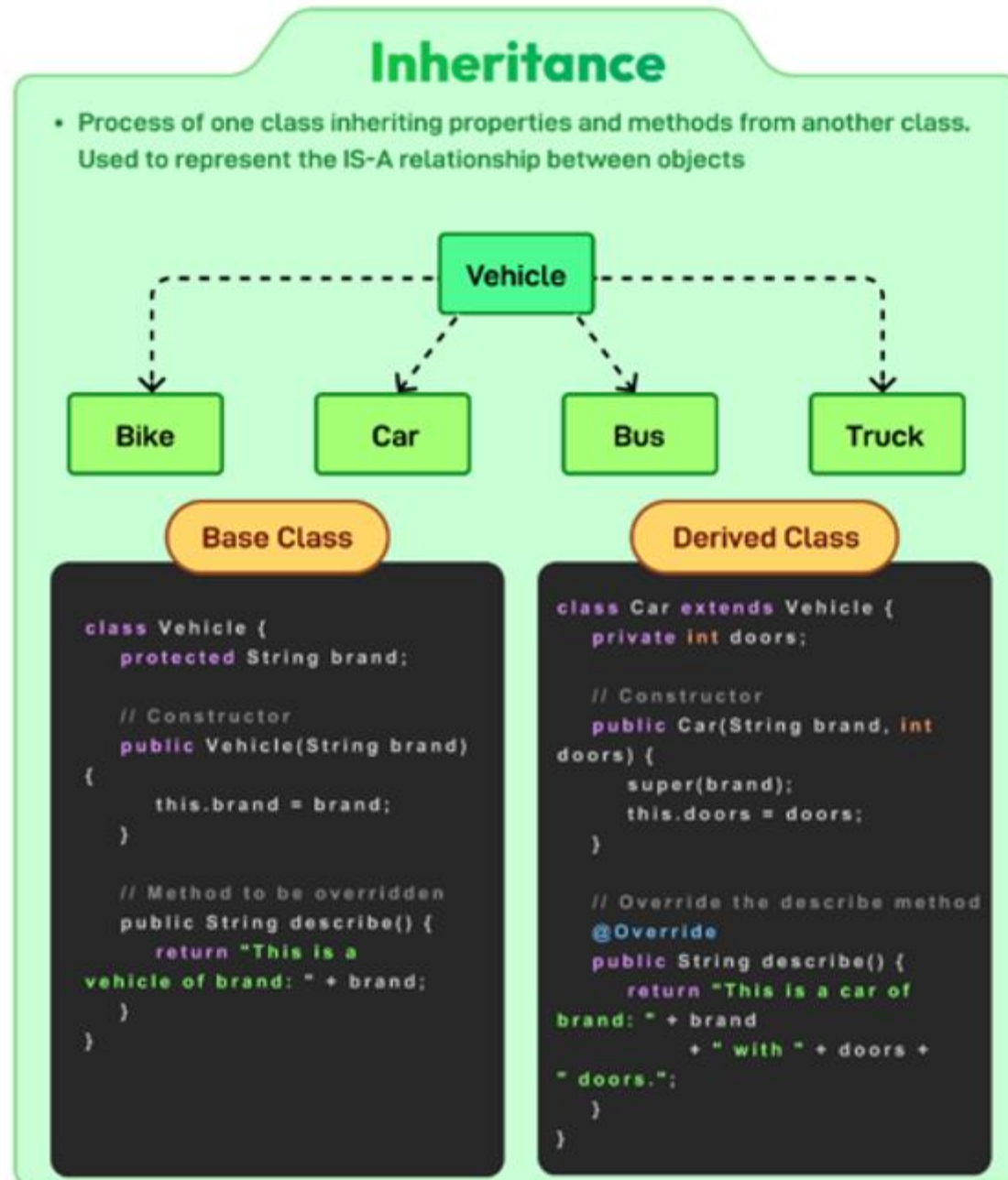


FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

3. Inheritance: – *Expanding Current Foundations*



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

4. Polymorphism: – *Multiple Forms, One Interface*

- **Definition:** Ability of the same function or method to behave differently depending on context.
- **Real-life example:**
 - The word “**run**”: you can *run a race*, *run a business*, or *run a program*
— same word, different meanings.
 - *In software:* a method draw() behaves differently for a Circle object vs. a Rectangle object.

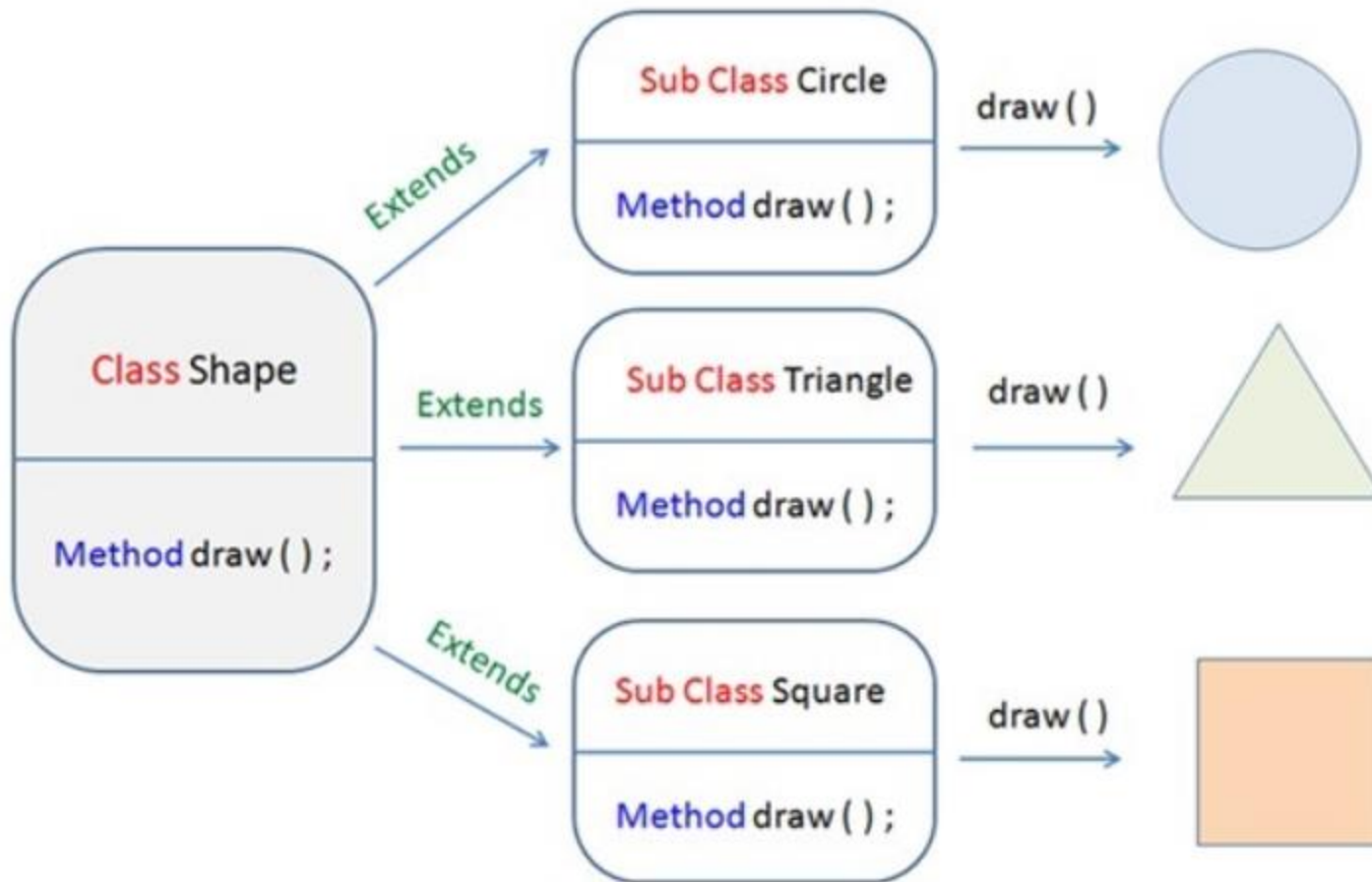
FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

4. Polymorphism: – *Multiple Forms, One Interface*

- Polymorphism means “many forms” — the ability of a single interface (method name) to represent different behaviors.
- Method Overloading allows multiple methods with the same name but different parameter lists in the **same class**.
- The compiler decides which version of the method to call at compile time based on the arguments passed.

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

4. Polymorphism: – *Multiple Forms, One Interface*



FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

4. Polymorphism: – *Multiple Forms, One Interface*

- **Method Overloading (Compile-time Polymorphism):** A method occurs when multiple methods in the same class share the same name but differ in parameters (number, type, or order).
- The compiler chooses which method is used based on the information that is provided.

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

4. Polymorphism: – *Multiple Forms, One Interface*

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

- 4. Polymorphism: – *Multiple Forms, One Interface*
 - **Method Overriding (Runtime Polymorphism)**: It is also known as runtime polymorphism, which occurs when a **subclass** provides its own unique implementation for a method stated in its superclass.

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

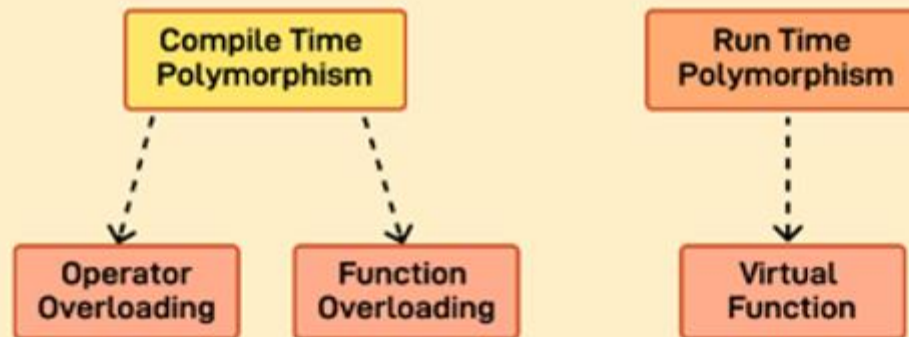
4. Polymorphism: – *Multiple Forms, One Interface*

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

Polymorphism

- Ability to perform many things in many ways. When two types share an inheritance chain, they can be used interchangeably with no errors. This also means using parents like their children



Definition

```
class Vehicle {  
    public void start() {  
        System.out.println("The  
vehicle starts.");  
    }  
}  
  
// Derived class 1  
class Car extends Vehicle {  
    @Override  
    public void start() {  
        System.out.println("The  
car starts with a key.");  
    }  
}
```

Usage

```
Vehicle myVehicle; // Reference  
of type  
  
// Assign Car object to the  
Vehicle reference  
myVehicle = new Car();  
  
myVehicle.start();
```

FOUR PILLARS OF OBJECT-ORIENTED PROGRAMMING

1. Encapsulation: Keep data safe inside a capsule.
2. Abstraction: Show the car's steering, hide the engine.
3. Inheritance: Children inherit traits from their parents.
4. Polymorphism: One word, many meanings (like "run").

To be continued...

```
If(anyQuestions)
{
    askNow();
}
else
{
    thankYou();
    submitAttendance();
    endClass();
}
```